

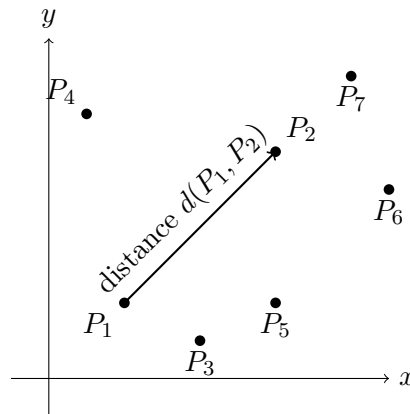
Lecture 9: Divide and Conquer

*Instructor: Sanjukta Roy**Scribe: R Vijayshri, Protyasha Kundu*

1 Closest pair of points

1.1 The Problem

We are given a set of n points, P , in $\mathbb{R}^2$ and we are asked to find the shortest distance between any two points in P .



1.2 The Solution

Let us first sort the points according to the x -co-ordinate.

Let x_m be the median of the sorted x co-ordinates.

Note: We modify x_m to be $x_m \pm \epsilon$ to make sure no point in P has its x co-ordinate as x_m .

1.2.1 Divide

Now we divide $P = P_1 \cup P_2$, where -

$$P_1 = \{(x, y) \in P : x < x_m\} \text{ and,}$$

$$P_2 = \{(x, y) \in P : x > x_m\}$$

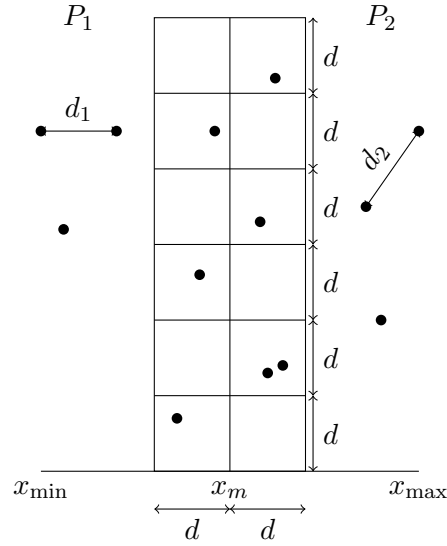
We solve the problem for P_1 and P_2 .

1.2.2 Conquer

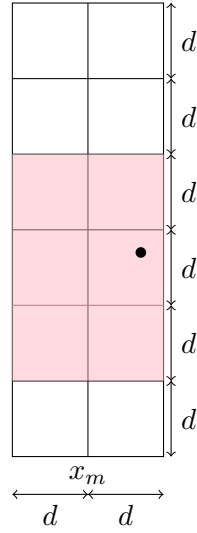
Let d_1, d_2 be the shortest distance between two points in P_1 and P_2 , respectively.

Let $d = \min(d_1, d_2)$. Since we have checked the distance between any two points in P_1 and any two points in P_2 , all that remains to be checked is if there are two points p_1, p_2 such that $p_1 \in P_1, p_2 \in P_2$ and $\text{dist}(p_1, p_2) < d$.

To do this, we consider a d -strip around x_m .



Notice that for each point in a square in the strip, if there is a point at a distance less than d from it, it has to be in one of the boxes surrounding it, which is a fixed number.



1.3 Pseudocode

```

1: procedure CLOSEST PAIR(P)
2:   Sort  $P$  according to the  $x$ -co-ordinate
3:    $x_m$  = the median of the  $x$  co-ordinates.
4:    $d_1$  = CLOSEST PAIR( $P_1$ )
5:    $d_2$  = CLOSEST PAIR( $P_2$ )
6:    $d$  =  $\min(\{d_i : d_i > 0 \text{ for } i = 1, 2\})$ 
7:   For each point in the  $d$ -strip, check constant number of boxes, update  $d$  as the
   current minimum.
8:   return final minimum distance.
9: end procedure

```

1.4 Runtime analysis

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

where, $O(n)$ is for checking the points in the d -strip, since there can be at-most n points in the d -strip.

Solving the recurrence, we get -

$$T(n) = O(n \log(n)),$$

including the time taken to sort the points, which also takes $O(n\log(n))$ time.

1.5 Proof of Correctness

We prove that this procedure yields the correct minimum distance by induction on $|P| = n$.

Base case: $n = 2$, in which case, the minimum distance is trivial.

Induction hypothesis: Assume that CLOSEST PAIR returns the minimum distance for a set of points P , with $|P| < n$.

Inductive step: For a set of points P with $|P| = n$, the algorithm divides P into P_1 and P_2 with $|P_1| = |P_2| = \frac{n}{2}$ and by inductive hypothesis we assume that CLOSEST PAIR recursively gives the correct minimum distances for P_1 and P_2 .

We defined $d = \min(\{d_i : d_i > 0 \text{ for } i = 1, 2\})$, where $d_i = \text{CLOSEST PAIR}(P_i)$.

Now, if there are two points x, y in P whose distance is less than d , then it must be that $x \in P_1, y \in P_2$ or vice-versa. Otherwise, our induction hypothesis is wrong.

So, if there exist such x and y , then they must lie in the d -strip. For each point in the d -strip, we are checking all the boxes surrounding its box, which contain points at a distance less than d . This is a constant number and we can easily find such points (if they exist) as P is sorted. Also, we note that a box can't contain more than 4 points, else we will have two points in the box at a distance less than d , violating the induction hypothesis.

Since we are checking for every point in the d -strip, the procedure will give the minimum distance value.

2 Karatsuba's Algorithm

Karatsuba's Algorithm uses Divide and Conquer paradigm to multiply two n digit number x and y .

2.1 The Setup

To multiply two n -digit numbers x and y , we split the numbers in half. Let

$$m = \lceil n/2 \rceil$$

. Then we can write x and y as

$$x = x_1 \cdot 10^m + x_0$$

$$y = y_1 \cdot 10^m + y_0$$

where

1. x_1 and y_1 are the most significant halves.
2. x_0 and y_0 are the least significant halves.

2.2 The Recursive Solution

The steps of recursive process

1. **Compute the high order product:**

$$z_2 = x_1 \cdot y_1$$

2. **Compute the low order product:**

$$z_0 = x_0 \cdot y_0$$

3. **Compute the combined product:**

$$z_1 = (x_1 + x_0) \cdot (y_1 + y_0) - z_2 - z_0$$

The term z_1 on expansion gives

$$(x_1 + x_0)(y_1 + y_0) - z_0 - z_2 = x_1y_1 + x_1y_0 + x_0y_1 + x_0y_0 - z_0 - z_2$$

On recombining each piece we get the final product:

$$xy = z_2 \cdot 10^{2m} + z_1 \cdot 10^m + z_0$$

This process is applied recursively to smaller parts until the numbers are small enough to multiply in $O(1)$ time.

2.3 Runtime Analysis

The recurrence relation for the algorithm's time complexity $T(n)$ is give as:

1. **Subproblems:** We make 3 recursive calls.
2. **Subproblem Size:** Each call multiplies numbers of size $n/2$
3. **Work to add, subtract, and shift:** Adding the results, doing the subtractions and shifting by 10^m and 10^{2m} takes linear time, $O(n)$ This gives the recurrence:

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

According to the Master Theorem, the time complexity is bounded by:

$$O(n^{\log_b a}) = O(n^{\log_2 3})$$

$$\log_2 3 \approx 1.585$$

Therefore, the runtime of the Karatsuba algorithm is $O(n^{1.585})$.

3 Tower of Hanoi

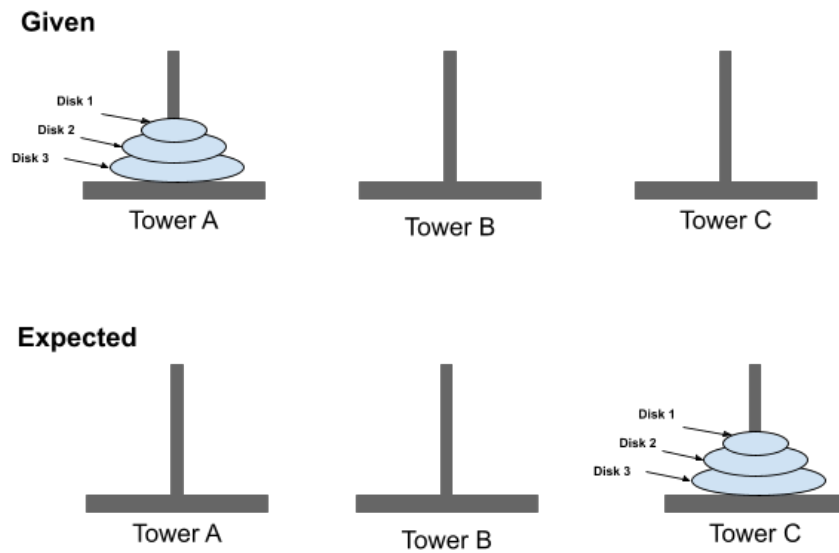


Figure 1: Tower of Hanoi

The Tower of Hanoi using Divide and Conquer paradigm.

3.1 The Setup

1. A,B and C be the three pegs.
2. n disks stacked on peg A with the smallest one on top.
3. We have to move the entire stack to peg C.

3.2 The Rules

1. We can only move one disk at a time.
2. We cannot move a larger disk on top of a smaller one.

3.3 The Solution

The solution is elegantly recursive in nature. To move n disks from peg A to peg C using peg B as an auxiliary peg:

1. Move $n - 1$ disk from peg A to peg B ($A \rightarrow B$) using peg C as the auxiliary peg.
2. Move the largest disk to C ($A \rightarrow C$)
3. Move $n - 1$ disk from peg B to peg C ($B \rightarrow C$) using peg A as the auxiliary peg.

3.4 Facts

No. of Disks	No. of Moves
1	1
2	3
3	7
.	.
.	.
.	.
n	$2^n - 1$

Why $2^n - 1$?

Every time we add a disk to $n - 1$ disks, we have to perform every step we did for $n - 1$ disks and move the new disk to a peg and move back $n - 1$ disks once again. Therefore, $(2^{n-1} - 1) + 1 + (2^{n-1} - 1) = 2^n - 1$ moves altogether for n disks. Therefore, the runtime analysis :

$$T(n) = 2T(n - 1) + O(1)$$